

VEKTÖR EMBEDDINGS VE SEMANTİK ARAMA

Stack Overflow Soru-Cevap Sistemi Üzerinden Çalışma Raporu

1. Semantik Arama ve Metin Gömme (Embeddings) Nedir?

Geleneksel arama motorları, kullanıcının yazdığı kelimeleri dökümanlardaki kelimelerle birebir eşleştiren **anahtar kelime aramasına (Keyword Search)** dayanır. Ancak bu yöntem, eş anlamlı kelimeleri, cümlenin bağlamını veya altında yatan asıl anlamı yakalayamaz. **Semantik Arama (Anlamsal Arama)** ise kelimelerin harflerine değil, taşıdığı **anlama ve niyete** odaklanır. Bu arama türünün temelinde **Metin Gömme (Text Embeddings)** teknolojisi yatar.

Metin gömme işlemi, bir cümlenin ya da paragrafın anlamsal içeriğini, bilgisayarların anlayabileceği yüksek boyutlu (genellikle 768 veya daha fazla boyutlu) yoğun bir **sayısal vektöre** dönüştürmektir. Anlamca benzer olan metinler vektör uzayında birbirine çok yakın konumlarda yer alırken, alakasız metinler birbirinden uzak konumlarda bulunur.

■ GÜNLÜK HAYAT ANALOJİSİ: Akıllı Kütüphaneci

Geleneksel anahtar kelime araması, bir kütüphaneye gidip kütüphaneciye içinde tam olarak 'elektrikli araba' kelimesi geçen kitapları sormanız gibidir. Kütüphaneci sadece kitap isimlerine bakar ve 'Sürdürülebilir Ulaşım' kitabını (içinde elektrikli arabalar detaylı anlatılsa bile) ismi tam uyuşmadığı için size vermez.

Semantik arama ise kütüphanecinin konuyu derinlemesine bilmesi gibidir. Siz ona 'bujisi ve egzozu olmayan, pil ile çalışan binek araçlar' dediğinizde, o sizin 'elektrikli araçları' kastettiğinizi anlar ve 'Geleceğin Ulaşım Teknolojileri' kitabını getirir. İşte metin gömme vektörleri, bilgisayara bu 'anlamsal kütüphaneci' yeteneğini kazandırır.

2. Grounding (Temellendirme) ve Halüsinasyon Önleme

Büyük Dil Modelleri (LLM'ler), eğitim verilerindeki genel bilgileri mükemmel şekilde saklarlar ancak **kurum içi özel verilere, güncel dökümanlara veya anlık değişen veritabanlarına** doğrudan erişemezler. Modelleri doğrudan eğitmek ya da ince ayar (Fine-tuning) yapmak maliyetli ve zahmetlidir. Ayrıca modeller bilmedikleri konularda mantıklı görünen ancak tamamen uydurma olan yanıtlar üretebilirler. Buna **Halüsinasyon (Hallucination)** denir.

Bunu çözmek için modeli harici bir bilgi tabanıyla (örneğin bir Stack Overflow veritabanı) ilişkilendirmeye **Grounding (Temellendirme)** adı verilir. Kullanıcının sorusuna en benzer dökümanlar semantik arama ile tespit edilir, bu dökümanlar dil modeline 'bağlam (context)'

olarak sunulur ve modelden yalnızca bu bilgilere dayanarak cevap üretmesi istenir. Bu yöntem, yanıtların doğruluğunu ve izlenebilirliğini (lineage) sağlar.

□ GÜNLÜK HAYAT ANALOJİSİ: Uzman Doktor ve Hasta Dosyası

Bir LLM, dünyadaki tüm tıp literatürünü ezbere bilen son derece zeki bir doktor gibidir. Ancak o doktor, kliniğe giren yeni bir hastanın (yani kurumunuzun özel verilerinin) geçmiş ameliyatlarını, alerjilerini ve özel durumlarını ezbere bilemez. Eğer hastayı geçmiş dosyalarına bakmadan tedavi etmeye çalışırsa yanlış ilaç verebilir (halüsinasyon).

Grounding ise, muayeneden hemen önce hastanın özel dosyasını (veri tabanını) bulup doktorun önüne koymaktır. Doktor genel tıp bilgisini (dil yeteneği) ve önüne konulan bu özel dosyayı (bağlam) harmanlayarak hatasız bir reçete yazar. Bu sayede reçetedeki her bilginin dosyanın hangi sayfasından geldiği takip edilebilir (traceability).

3. Soru-Cevap Sisteminin Mimarisi ve Kod Akışı

Raporun bu bölümünde, Google Cloud Vertex AI SDK'sı ve Python kullanarak adım adım uçtan uca çalışan bir Stack Overflow Soru-Cevap sisteminin inşası teknik kod örnekleriyle incelenmektedir. Sistemimiz 2.000 satırlık gerçek bir Python Stack Overflow gönderi veritabanını kullanmaktadır.

Adım 3.1: SDK Kurulumu ve Veri Tabanının Yüklenmesi

İlk olarak sistem yetkilendirmesi yapılır, bölge ayarlanır ve Vertex AI SDK'sı başlatılır. Veri tabanımızda Stack Overflow üzerindeki soru başlıkları, kullanıcıların kabul ettiği çözümler ve programlama dili etiketleri bulunmaktadır. Veriler Pandas kütüphanesi ile belleğe yüklenir.

```
import pandas as pd
import vertexai
from vertexai.language_models import TextEmbeddingModel, TextGenerationModel

# Vertex AI ve Bölge Tanımlaması
vertexai.init(project="proje-id-niz", location="us-central1")

# Stack Overflow Veri Setini Yükleme (2,000 Gönderi)
# Sütunlar: 'input_text' (soru başlığı), 'output_text' (çözüm), 'category' (dil)
so_database = pd.read_csv("stack_overflow_posts.csv")
print("Veri Seti Boyutu:", so_database.shape) # Çıktı: (2000, 3)
```

Adım 3.2: Metinleri Vektörleştirme (Embedding Coğrafyası)

Her bir Stack Overflow sorusunu sayısal temsile çevirmek için Vertex AI **textembedding-gecko@001** modeli kullanılır. API limitlerini yönetmek amacıyla toplu işlem yapan özel bir fonksiyon ('encode_text_to_embedding_batched') kullanılır ve elde edilen vektörler veritabanına sütun olarak eklenir.

```
# Gömme Modelinin Yüklenmesi
embedding_model = TextEmbeddingModel.from_pretrained("textembedding-gecko@001")

# Vektörlerin veritabanına yeni bir sütun olarak eklenmesi
# (Eğitim ortamında API çağrılarından tasarruf için önceden pickle ile kaydedilmiş veriler yüklenmiştir)
import pickle
with open("embeddings.pkl", "rb") as f:
    precomputed_embeddings = pickle.load(f)

so_database['embeddings'] = precomputed_embeddings
print(so_database.head(2)) # 'embeddings' sütunu başarıyla eklendi.
```

4. Doğru Yakınlık Ölçümünün Seçilmesi (Mesafe Metrikleri)

Kullanıcı bir soru sorduğunda, bu sorunun vektörü ile veritabanımızdaki 2.000 sorunun vektörü arasındaki anlamsal benzerliği ölçmek için matematiksel formüller kullanırız. En yaygın kullanılan üç metrik şunlardır:

Mesafe Metriği	Matematiksel Açıklama	Kullanım Alanı ve Karakteristiği
Cosine Similarity (Kosinüs Benzerliği)	İki vektör arasındaki açının kosinüsünü ölçer. Vektör uzunluğundan (metin boyutundan) bağımsızdır.	Metin benzerliği, semantik arama ve genel NLP görevleri için en ideal metriktir.
Euclidean Distance (Öklid Mesafesi - L2)	İki vektörün uç noktaları arasındaki geometrik düz çizgisel mesafeyi hesaplar.	Genel veri kümeleme (K-Means) ve metin dışı mesafe hesaplamalarında tercih edilir.
Dot Product (Nokta Çarpımı)	Vektörlerin hem yönünü hem de büyüklüğünü dikkate alır.	Eğer vektörler normalize edilmişse (büyüklüğü 1 ise), kosinüs benzerliği ile tamamen aynı sonucu verir.

5. Benzerlik Araması ve En Yakın Komşuyu Bulma

Kullanıcı 'how to concatenate data frames in pandas?' sorusunu sorduğunda, bu sorguyu vektöre dönüştürüp **scikit-learn** kütüphanesi kullanarak tüm veritabanı ile karşılaştırırız. En yüksek kosinüs benzerliğine sahip gönderinin indeksi tespit edilir.

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# 1. Kullanıcı sorusunu vektörleştirme
user_query = "how to concatenate data frames in pandas?"
query_vector = embedding_model.get_embeddings([user_query])[0].values

# 2. Sorgu vektörünü tüm veritabanı vektörleriyle karşılaştırma (Kosinüs Benzerliği)
# cosine_similarity fonksiyonu 2D dizi bekler, bu yüzden listeye sarılır
similarities = cosine_similarity([query_vector], list(so_database['embeddings']))

# 3. En yüksek benzerlik skoruna sahip satırı bulma
most_similar_idx = np.argmax(similarities)
matched_post = so_database.iloc[most_similar_idx]

print("En Yakın Eşleşen Soru:", matched_post['input_text'])
# Çıktı: "how to concatenate objects in pandas" gibi anlamsal olarak çok benzer bir soru.
```

6. LLM ile Temellendirilmiş (Grounded) Cevap Üretimi

Bulduğumuz ham Stack Overflow cevabı genellikle okunması zor kod kalıntıları veya eksik açıklamalar barındırabilir. Bunun yerine, Vertex AI **text-bison@001** modeline orijinal kullanıcı sorusunu ve bulduğumuz en yakın eşleşen dökümanı **bağlam (context)** olarak vererek, kullanıcıya temiz ve anlaşılır bir açıklama hazırlatırız.

```
# Dil modelinin yüklenmesi
generation_model = TextGenerationModel.from_pretrained("text-bison@001")

# Prompt Tasarımı ve Temellendirme (Grounding) Direktifi
prompt = f"""
Aşağıdaki bağlam (context) alanındaki bilgileri kullanarak kullanıcının sorusuna anlaşılır ve samimi bir cevap üret.
Eğer bağlamda soruya yönelik faydalı bir bilgi bulunmuyorsa, kesinlikle dışarıdan bilgi uydurma ve tam olarak şu c
'I couldn't find a match in the document database for your query.'

Bağlam Bilgisi:
Soru: {matched_post['input_text']}
Cevap: {matched_post['output_text']}

Kullanıcının Sorduğu Soru:
{user_query}
"""

# Cevabın üretilmesi (Düşük sıcaklık/temperature ile kararlılık sağlanır)
response = generation_model.predict(prompt, temperature=0.2, max_output_tokens=512)
print("Sistem Yanıtı (Markdown formatında):\n", response.text)
```

Sınır Dışı (Out-of-Domain) Sorguların Yönetilmesi

Sistemimize Python ile tamamen alakasız olan 'how to make the perfect lasagna?' (en mükemmel lazanya nasıl yapılır?) gibi bir soru geldiğinde, veritabanındaki en yakın vektör bile lazanya ile ilgili olamaz. Prompt içerisine eklediğimiz koruyucu kural sayesinde model, bilmediği bu konuda uydurma bilgiler vermez ve güvenli bir şekilde **'I couldn't find a match in the document database for your query.'** yanıtını döner. Bu yöntem, ticari sistemlerin güvenliği için hayati bir adımdır.

7. Büyük Veride Ölçeklenebilirlik ve ScaNN Algoritması

Küçük ölçekli projelerde (örneğin 2.000 vektör içeren veritabanımızda) her sorguyu tüm vektörlerle tek tek karşılaştırmak (**Exhaustive Search**) milisaniyeler sürer. Ancak veritabanınızda milyonlarca veya milyarlarca döküman olduğunda bu yöntem sistemi kilitler ve sorgu süresi kabul edilemez derecede uzar.

Bunun önüne geçmek için **Yaklaşık En Yakın Komşu (Approximate Nearest Neighbors - ANN)** algoritmaları kullanılır. Google tarafından geliştirilen açık kaynaklı **ScaNN (Scalable Nearest Neighbors)** kütüphanesi, vektör uzayını akıllıca bölerek milyarlarca veri içinde mikro saniyeler seviyesinde benzerlik araması yapabilmektedir.

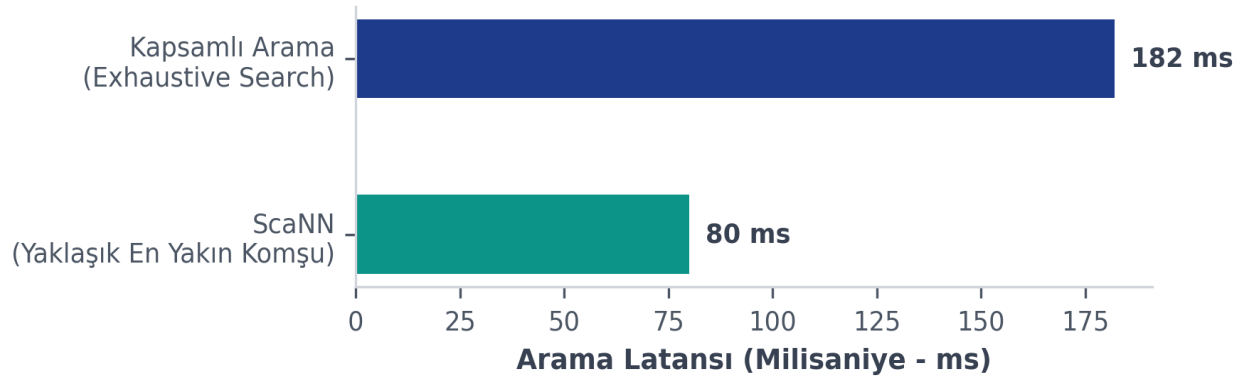
```
import scann
import time

# ScaNN Vektör İndeksi Oluşturma
# embeddings dizisini hızlı arama için indeksliyoruz
scann_index = scann.scann_ops_pybind.builder(
    np.array(list(so_database['embeddings'])),
    num_neighbors=1,
    distance_measure="dot_product"
).tree(
    num_leaves=100,          # Arama ağacındaki yaprak sayısı
    num_leaves_to_search=10, # Aranacak yaprak sayısı (hız/doğruluk dengesi)
    training_sample_size=2000
).score_ah(
    2                          # Asimetrik Hashing aktivasyonu
).build()

# ScaNN ile Hızlı Arama ve Latans Ölçümü
start_time = time.time()
neighbors, distances = scann_index.search(np.array(query_vector))
scann_latency = (time.time() - start_time) * 1000 # Milisaniyeye çevirme

print(f"ScaNN Arama Süresi: {scann_latency:.2f} ms")
```

Arama Algoritmalarının Performans Karşılaştırması



Grafik Açıklaması: Stack Overflow veri tabanında yapılan aramalarda ScaNN algoritması exhaustive arama yöntemine göre yaklaşık **%56 daha hızlı** sonuç üretmiştir. Büyük veri kümelerinde bu fark katlanarak açılmaktadır.

8. Sonuç ve Genel Değerlendirme

Bu çalışmada, Vertex AI ve Metin Gömme (Embeddings) teknolojisi entegre edilerek, bilgi doğruluğu yüksek ve halüsinasyondan arındırılmış bir Soru-Cevap sisteminin prototipi kurulmuştur. Sistem, semantik yakınlık hesaplamalarında **Kosinüs Benzerliği'ni** kullanarak kelime benzerliklerinin ötesine geçip anlamsal eşleştirmeler yapabilmektedir. Büyük verilerle çalışırken sisteme entegre edilen **ScaNN (ANN)** algoritması, doğruluktan ödün vermeden sistemin arama latansını dramatik şekilde düşürmektedir. Bu metodoloji, kurumsal bilgi yönetim sistemlerinde, müşteri destek botlarında ve her türlü RAG (Retrieval-Augmented Generation) mimarisinde güvenle uygulanabilir.